



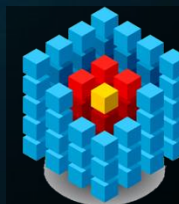
# Inteligencia Artificial y Aprendizaje Automático

Sesión 6: Redes Neuronales  
Optimización de rendimiento

Presentan:

Dr. Ulises Olivares Pinto

Fernando Ramírez González



HPC  
DATA LAB

# CNNs para clasificación



```
tf.keras.layers.Conv2D
```



```
tf.keras.activations.*
```



```
tf.keras.layers.MaxPool2D
```

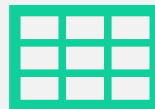


**1. Convolución:** Aplicar filtros para generar mapas de características



**2. Introducir no linealidad (función de activación):**

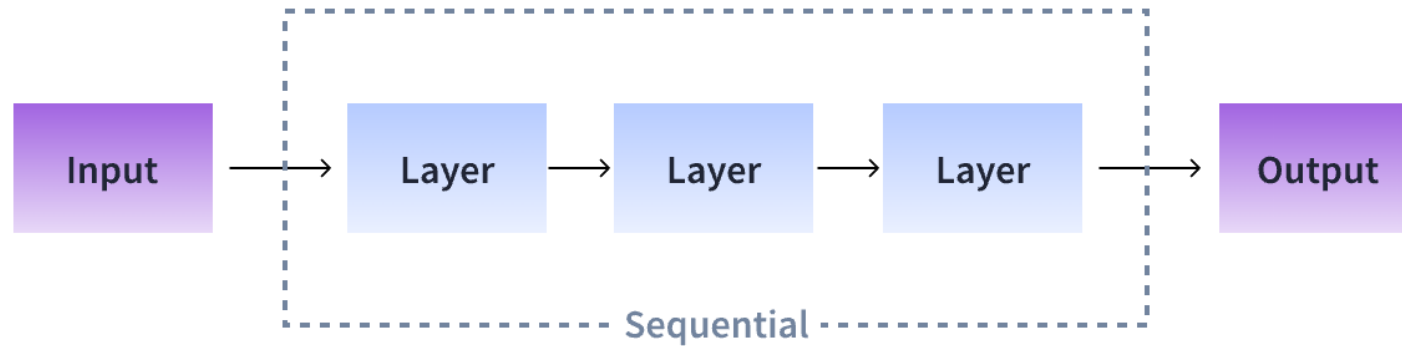
Usualmente se utiliza la función RELU



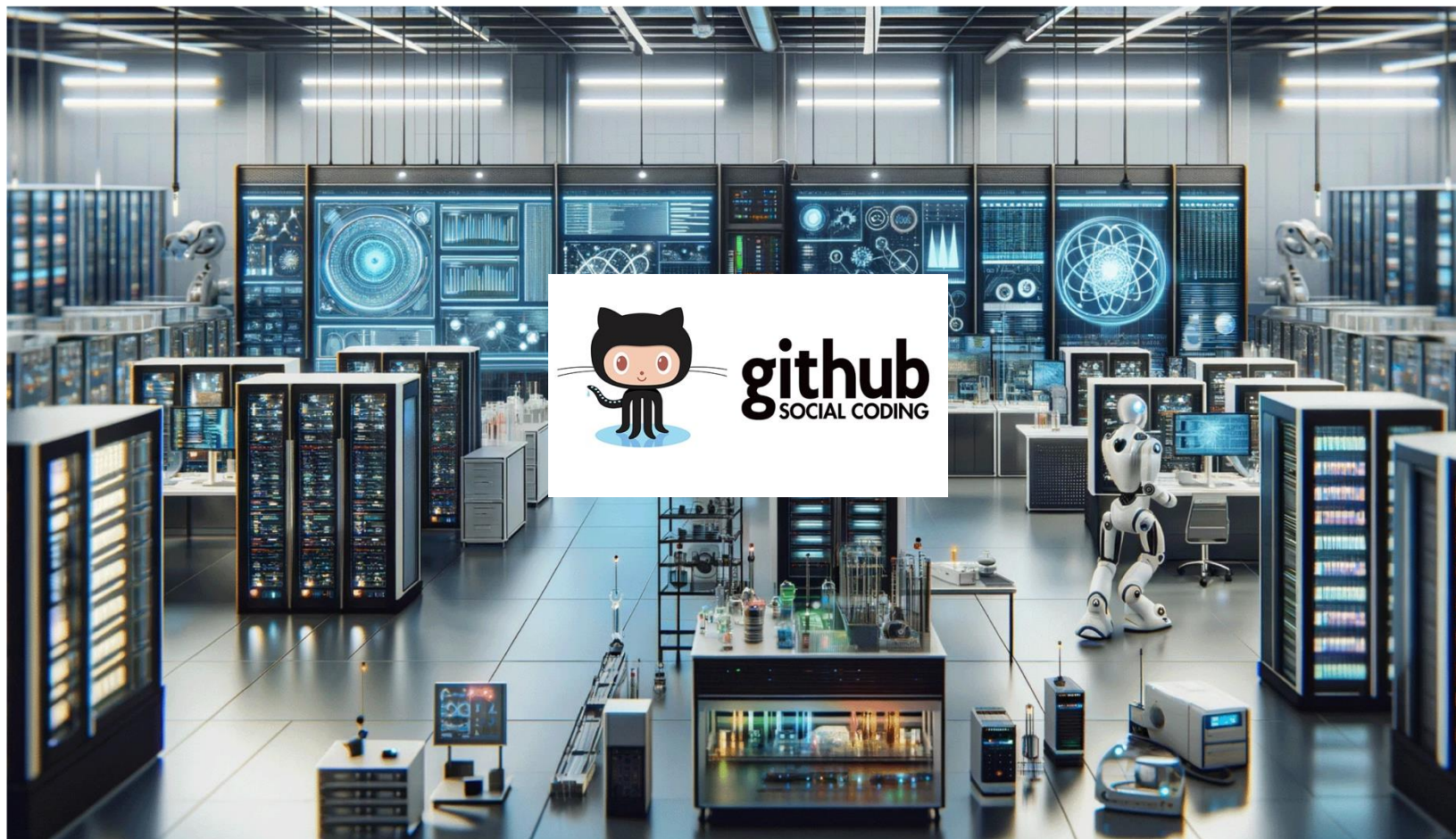
**3. Agrupación (Pooling):** Agrupar los datos de cada mapa de características.

# Apilando capas Convolucionales

 Keras Sequential API



```
1 model = models.Sequential()
2 model.add(layers.Conv2D(32, (3, 3), activation='relu', input_shape=(32, 32, 3)))
3 model.add(layers.MaxPooling2D((2, 2)))
4 model.add(layers.Conv2D(64, (3, 3), activation='relu'))
5 model.add(layers.MaxPooling2D((2, 2)))
6 model.add(layers.Conv2D(64, (3, 3), activation='relu'))
7
8 # Añadiendo capas densas en el final
9 model.add(layers.Flatten())
10 model.add(layers.Dense(64, activation='relu'))
11 model.add(layers.Dense(10))
```





# TensorFlow

## Demo Base: CNN

## y CIFAR 10

CIFAR - 10

airplane

automobile

bird

cat

deer

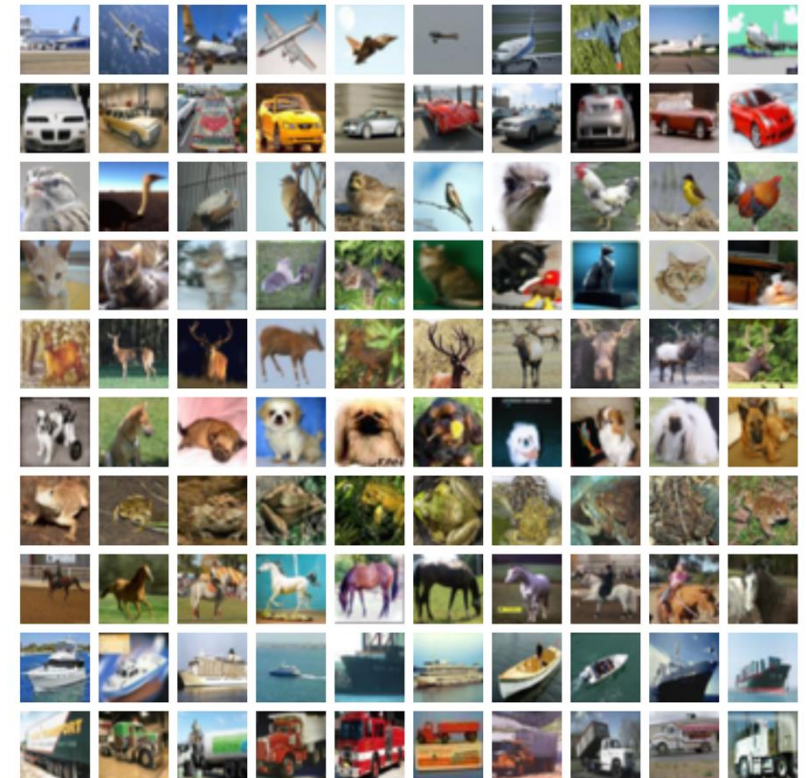
dog

frog

horse

ship

truck





+ Code + Text

Reconnect T4 ▼



# Inteligencia Artificial - Redes Neuronales convolucionales

## Clasificación de imágenes usando CNNs y el set de datos CIFAR 10

Autor: Ulises Olivares Pinto

7 de marzo de 2024

## 1. Importar Bibliotecas Necesarias Código Fuente

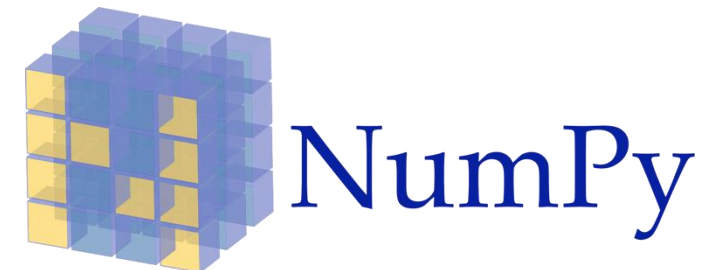
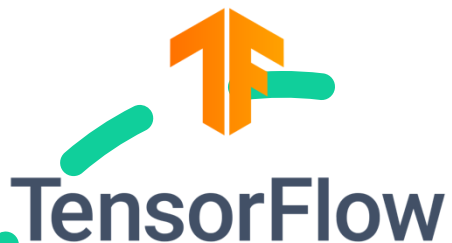
```
[ ] 1 import tensorflow as tf
    2 from tensorflow.keras import datasets, layers, models
    3 import matplotlib.pyplot as plt # Librería para generar gráficas
    4 import numpy as np # Librería para manejar arreglos en python
    5 from sklearn.metrics import confusion_matrix # Librería para crear matriz de confusión
    6 import seaborn as sns # Librería para generar mapa de calor
```

# 1. Importar librerías



## 1. Importar Bibliotecas Necesarias

```
[ ] 1 import tensorflow as tf
    2 from tensorflow.keras import datasets, layers, models
    3 import matplotlib.pyplot as plt # Librería para generar gráficas
    4 import numpy as np # Librería para manejar arreglos en python
    5 from sklearn.metrics import confusion_matrix # Librería para crear matriz de confusión
    6 import seaborn as sns # Librería para generar mapa de calor
```



## 2. Cargar datos

### 2. Cargar el Conjunto de Datos CIFAR-10

```
[11] 1 (train_images, train_labels), (test_images, test_labels) = datasets.cifar10.load_data()  
2 train_images, test_images = train_images / 255.0, test_images / 255.0  
3 class_names = ['airplane', 'automobile', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse', 'ship', 'truck']  
4 total_imgs = len(train_images) + len(test_images)  
5 print("Imágenes totales:", total_imgs)  
6 print("Imágenes para entrenamiento:", len(train_images), "~", "{:.2f}".format((len(train_images)/total_imgs)*100), "%")  
7 print("Imágenes para prueba:", len(test_images), "~", "{:.2f}".format((len(test_images)/total_imgs)*100), "%")  
8  
9
```

```
Imágenes totales: 60000  
Imágenes para entrenamiento: 50000 ~ 83.33 %  
Imágenes para prueba: 10000 ~ 16.67 %
```

## Datasets

[Overview](#)
[Catalog](#)
[Community Catalog](#)
[Guide](#)
[API](#)

TFDS now supports the [Croissant](#) format! Read the [documentation](#) to know more.

TensorFlow Datasets: a collection of ready-to-use datasets.

TensorFlow Datasets is a collection of ready-to-use, with TensorFlow or other Python ML frameworks, such as Jax. All datasets are exposed as `tf.data.Datasets`, enabling easy-to-use and high performance input pipelines. To get started see the [guide](#) and our [list of datasets](#).

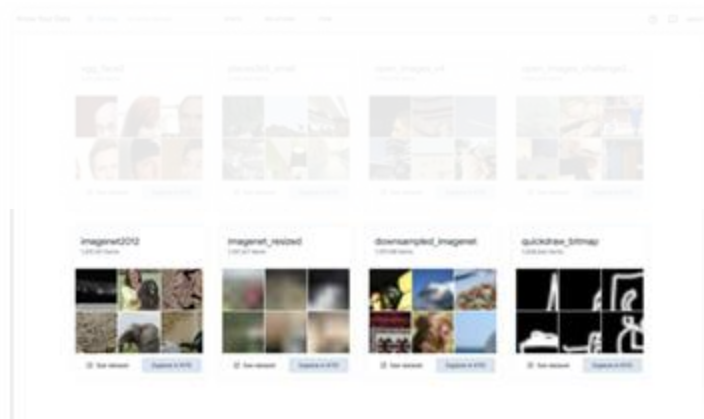
<https://www.tensorflow.org/datasets>

```
import tensorflow as tf
import tensorflow_datasets as tfds

# Construct a tf.data.Dataset
ds = tfds.load('mnist', split='train', shuffle_files=True)

# Build your input pipeline
ds = ds.shuffle(1024).batch(32).prefetch(tf.data.AUTOTUNE)
for example in ds.take(1):
    image, label = example["image"], example["label"]
```

RUN IN A [COLAB](#) NOTEBOOK



# Tf.datasets.cifar10.loaddata()

TFDS now supports the [Croissant](#) format! Read the [documentation](#) to know more.

TensorFlow > Resources > Datasets > Catalog

Was this helpful?

cifar10

Visualization: [Explore in Know Your Data](#)

Description:

The CIFAR-10 dataset consists of 60000 32x32 colour images in 10 classes, with 6000 images per class. There are 50000 training images and 10000 test images.

Additional Documentation: [Explore on Papers With Code](#)

Homepage: <https://www.cs.toronto.edu/~kriz/cifar.html>

Source code: [tfds.image\\_classification.Cifar10](#)

Versions:

- 3.0.2 (default): No release notes.

Download size: 162.17 MiB

Dataset size: 132.40 MiB

Auto-cached ([documentation](#)): Yes

Splits:

| Split   | Examples |
|---------|----------|
| 'test'  | 10,000   |
| 'train' | 50,000   |

Feature structure:

```
FeaturesDict({
  'id': Text(shape=(), dtype=string),
  'image': Image(shape=(32, 32, 3), dtype=uint8),
  'label': ClassLabel(shape=(), dtype=int64, num_classes=10),
})
```

## • Feature documentation:

| Feature      | Class      | Shape       | Dtype  | Description |
|--------------|------------|-------------|--------|-------------|
| FeaturesDict |            |             |        |             |
| id           | Text       |             | string |             |
| image        | Image      | (32, 32, 3) | uint8  |             |
| label        | ClassLabel |             | int64  |             |

• **Supervised keys** (See [as\\_supervised doc](#)): ('image', 'label')

• **Figure** ([tfds.show\\_examples](#)):

horse (7)   ship (8)   deer (4)

deer (4)   frog (6)   dog (5)

bird (2)   truck (9)   frog (6)

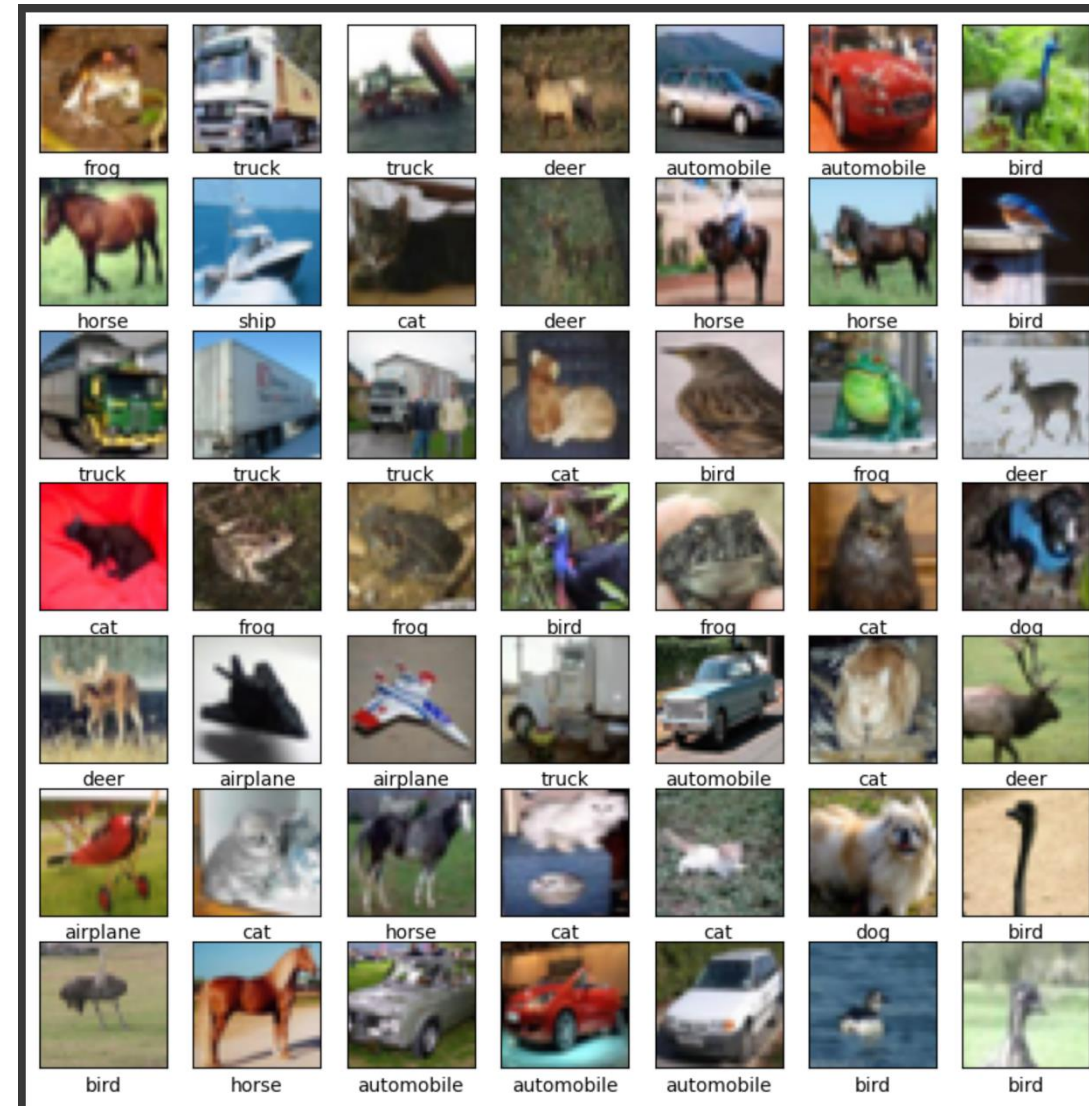
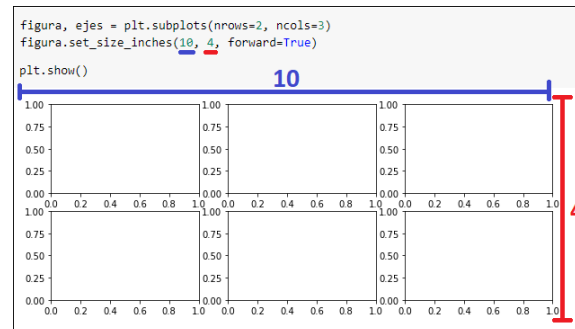
# Verificar Datos

32 x 32 pixeles

## 3. Verificar el Conjunto de Datos

```
[ ] 1
    2 plt.figure(figsize=(10,10))
    3 for i in range(49):
    4     plt.subplot(7,7,i+1)
    5     plt.xticks([])
    6     plt.yticks([])
    7     plt.grid(True)
    8     plt.imshow(train_images[i], cmap=plt.cm.binary)
    9     plt.xlabel(class_names[train_labels[i][0]])
10
11 plt.show()
```

matplotlib



# Construir modelo

## 4. Construir el Modelo

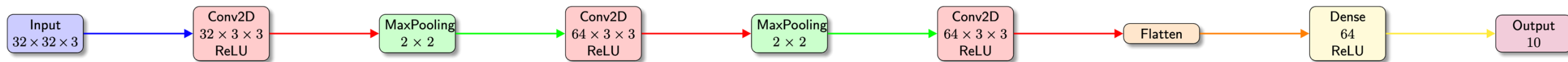
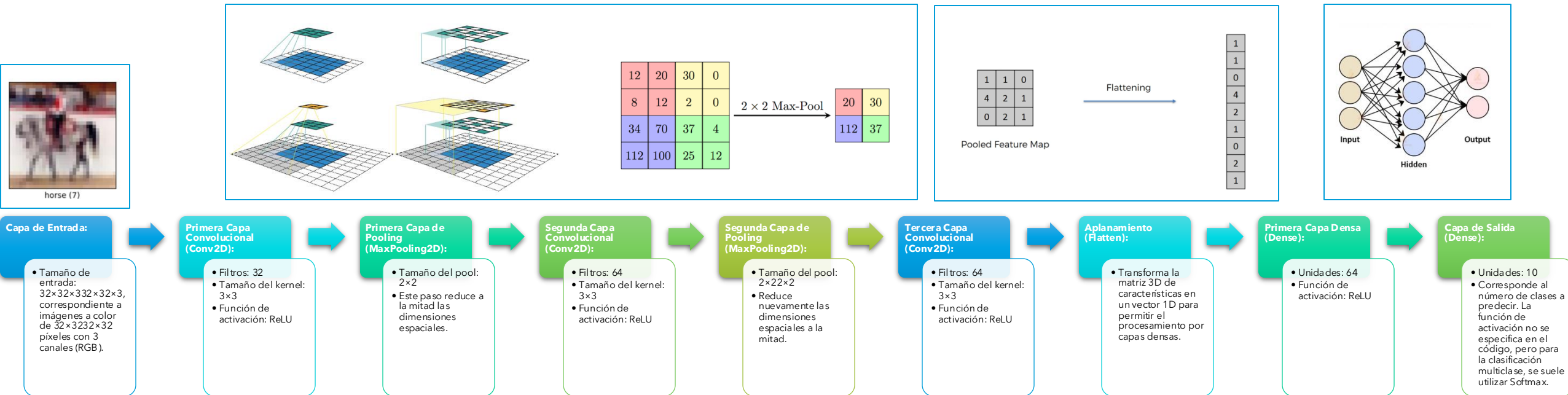


```
1
2 model = models.Sequential([
3     layers.Conv2D(32, (3, 3), activation='relu', input_shape=(32, 32, 3)),
4     layers.MaxPooling2D((2, 2)),
5     layers.Conv2D(64, (3, 3), activation='relu'),
6     layers.MaxPooling2D((2, 2)),
7     layers.Conv2D(64, (3, 3), activation='relu'),
8     layers.Flatten(),
9     layers.Dense(64, activation='relu'),
10    layers.Dense(10)
11 ])
12
13 model.summary()
14
```

Model: "sequential"

| Layer (type)                         | Output Shape       | Param # |
|--------------------------------------|--------------------|---------|
| =====                                |                    |         |
| conv2d (Conv2D)                      | (None, 30, 30, 32) | 896     |
| max_pooling2d (MaxPooling2D)         | (None, 15, 15, 32) | 0       |
| conv2d_1 (Conv2D)                    | (None, 13, 13, 64) | 18496   |
| max_pooling2d_1 (MaxPooling2D)       | (None, 6, 6, 64)   | 0       |
| conv2d_2 (Conv2D)                    | (None, 4, 4, 64)   | 36928   |
| flatten (Flatten)                    | (None, 1024)       | 0       |
| dense (Dense)                        | (None, 64)         | 65600   |
| dense_1 (Dense)                      | (None, 10)         | 650     |
| =====                                |                    |         |
| Total params: 122570 (478.79 KB)     |                    |         |
| Trainable params: 122570 (478.79 KB) |                    |         |
| Non-trainable params: 0 (0.00 Byte)  |                    |         |

# Arquitectura Simple



# Compilar y entrenar el modelo

## 5. Compilar y Entrenar el Modelo

```
[ ] 1
    2 model.compile(optimizer='adam',
    3                 loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True),
    4                 metrics=['accuracy'])
    5
    6 history = model.fit(train_images, train_labels, epochs=10,
    7                     validation_data=(test_images, test_labels))
    8
```

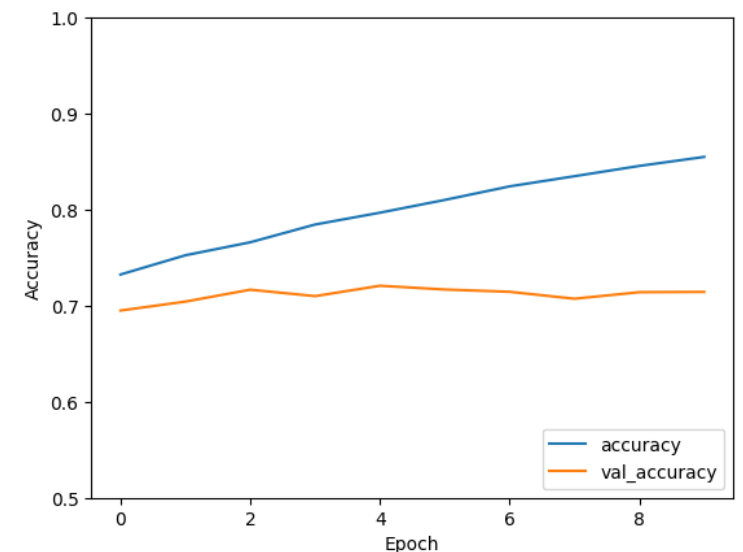
```
Epoch 1/10
1563/1563 [=====] - 67s 42ms/step - loss: 1.5204 - accuracy: 0.4449 - val_loss: 1.4127 - val_accuracy: 0.5092
Epoch 2/10
1563/1563 [=====] - 64s 41ms/step - loss: 1.1654 - accuracy: 0.5847 - val_loss: 1.0655 - val_accuracy: 0.6198
Epoch 3/10
1563/1563 [=====] - 63s 40ms/step - loss: 1.0130 - accuracy: 0.6434 - val_loss: 1.0156 - val_accuracy: 0.6401
Epoch 4/10
1563/1563 [=====] - 63s 40ms/step - loss: 0.9175 - accuracy: 0.6788 - val_loss: 0.9675 - val_accuracy: 0.6645
Epoch 5/10
1563/1563 [=====] - 63s 40ms/step - loss: 0.8505 - accuracy: 0.7020 - val_loss: 0.9279 - val_accuracy: 0.6788
Epoch 6/10
1563/1563 [=====] - 66s 42ms/step - loss: 0.7952 - accuracy: 0.7205 - val_loss: 0.9327 - val_accuracy: 0.6774
Epoch 7/10
1563/1563 [=====] - 68s 43ms/step - loss: 0.7477 - accuracy: 0.7392 - val_loss: 0.8764 - val_accuracy: 0.6943
Epoch 8/10
1563/1563 [=====] - 66s 42ms/step - loss: 0.7007 - accuracy: 0.7544 - val_loss: 0.8811 - val_accuracy: 0.7067
Epoch 9/10
1563/1563 [=====] - 68s 44ms/step - loss: 0.6583 - accuracy: 0.7705 - val_loss: 0.9269 - val_accuracy: 0.6955
Epoch 10/10
1563/1563 [=====] - 67s 43ms/step - loss: 0.6219 - accuracy: 0.7808 - val_loss: 0.9022 - val_accuracy: 0.6986
```

# Evaluar el modelo

## 6. Evaluar el Modelo

```
[15] 1
2 plt.plot(history.history['accuracy'], label='accuracy')
3 plt.plot(history.history['val_accuracy'], label = 'val_accuracy')
4 plt.xlabel('Epoch')
5 plt.ylabel('Accuracy')
6 plt.ylim([0.5, 1])
7 plt.legend(loc='lower right')
8 plt.show()
9
10 plt.plot(history.history['loss'], label='Training loss')
11 plt.plot(history.history['val_loss'], label='Validation loss')
12 plt.xlabel('Epoch')
13 plt.ylabel('Loss')
14 plt.ylim([0, 1]) # Ajuste el rango según sea necesario para su conjunto de datos y modelo
15 plt.legend(loc='upper right')
16 plt.title('Training and Validation Loss')
17 plt.show()
18
19 test_loss, test_acc = model.evaluate(test_images, test_labels, verbose=2)
20 print("presición sobre el set de datos de prueba:", test_acc)
21
```

313/313 - 4s - loss: 0.9580 - accuracy: 0.7144 - 4s/epoch - 13ms/step  
presición sobre el set de datos de prueba: 0.7143999934196472

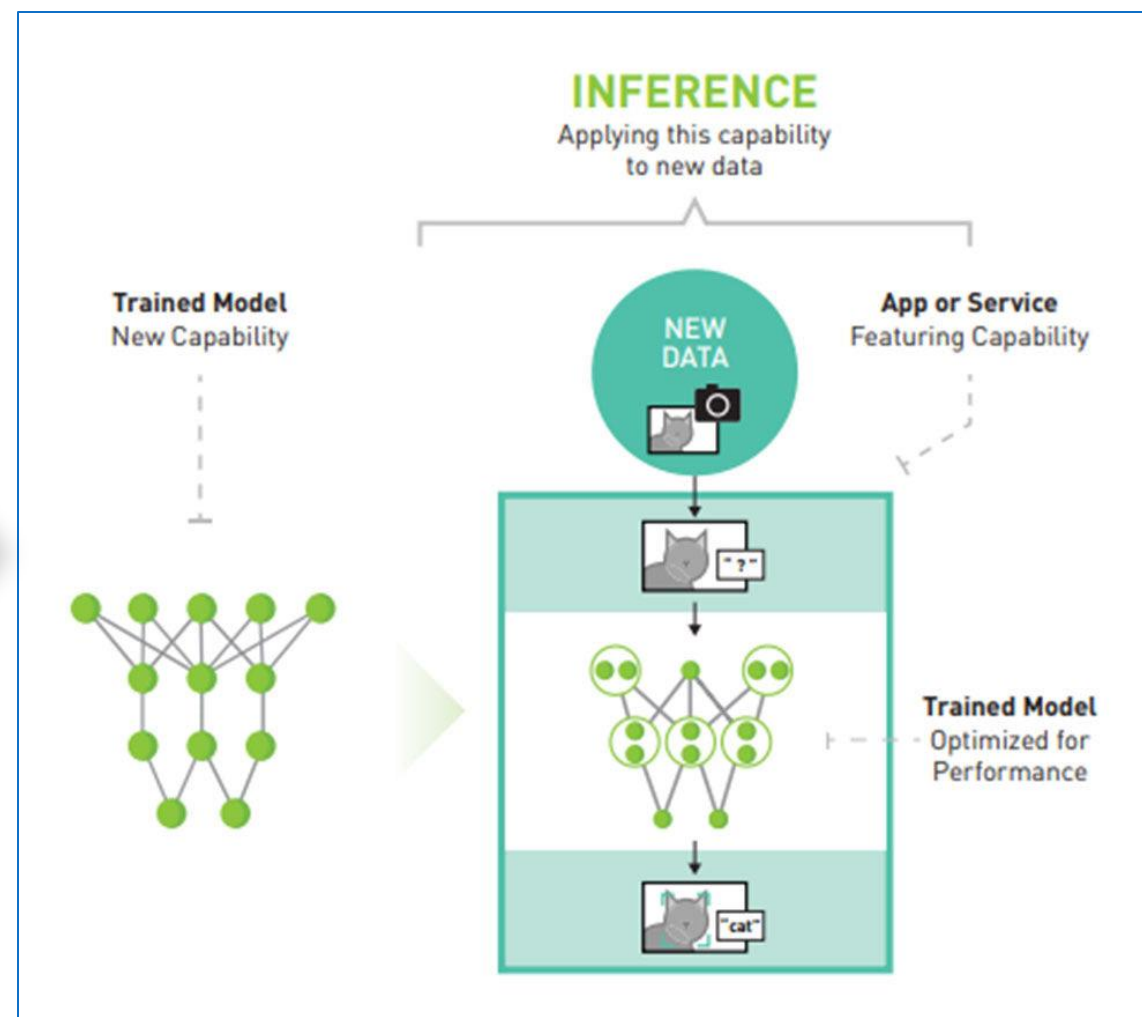
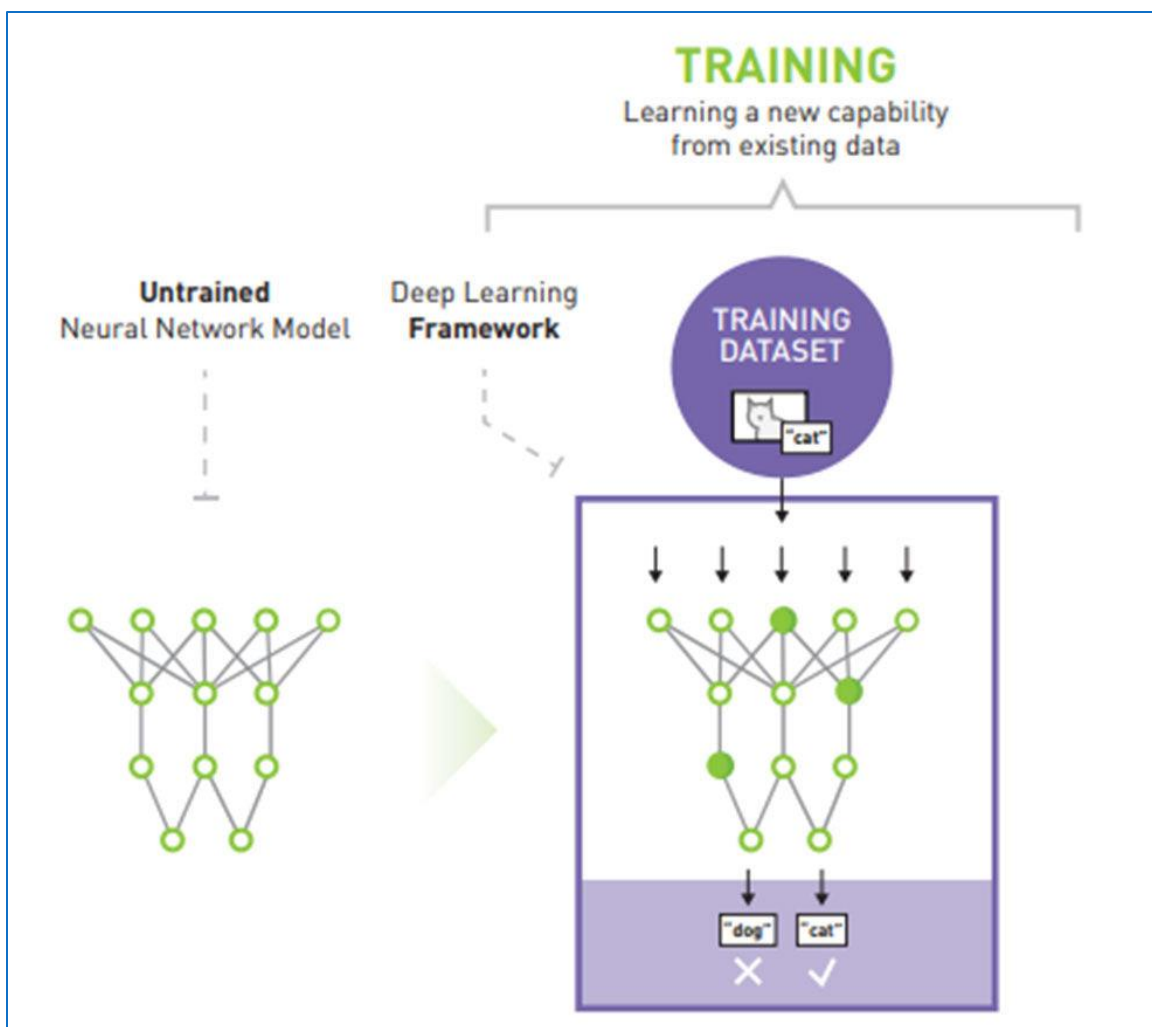




313/313 [=====] - 4s 13ms/step



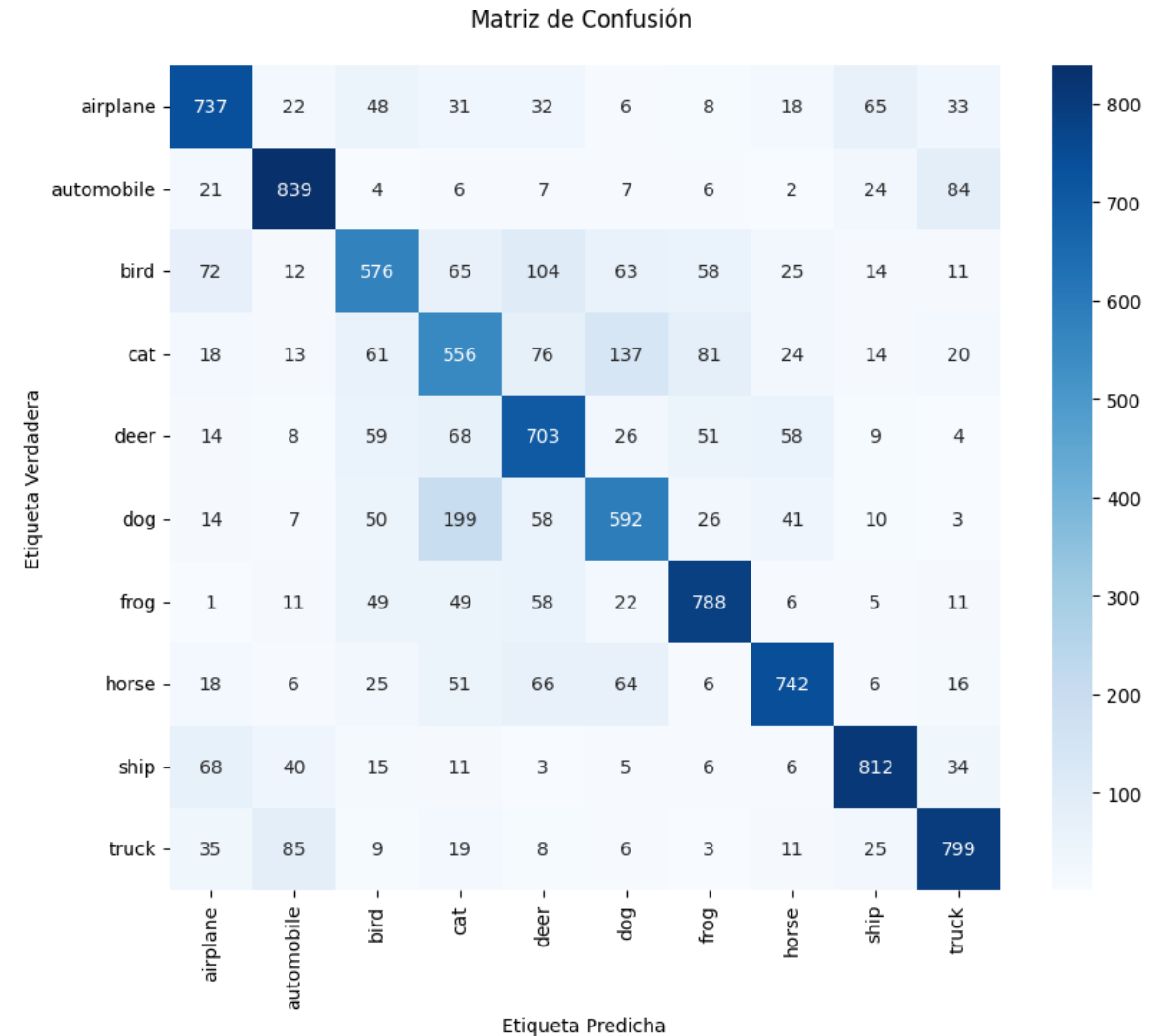
# Entrenamiento vs inferencia



# Matriz de confusión

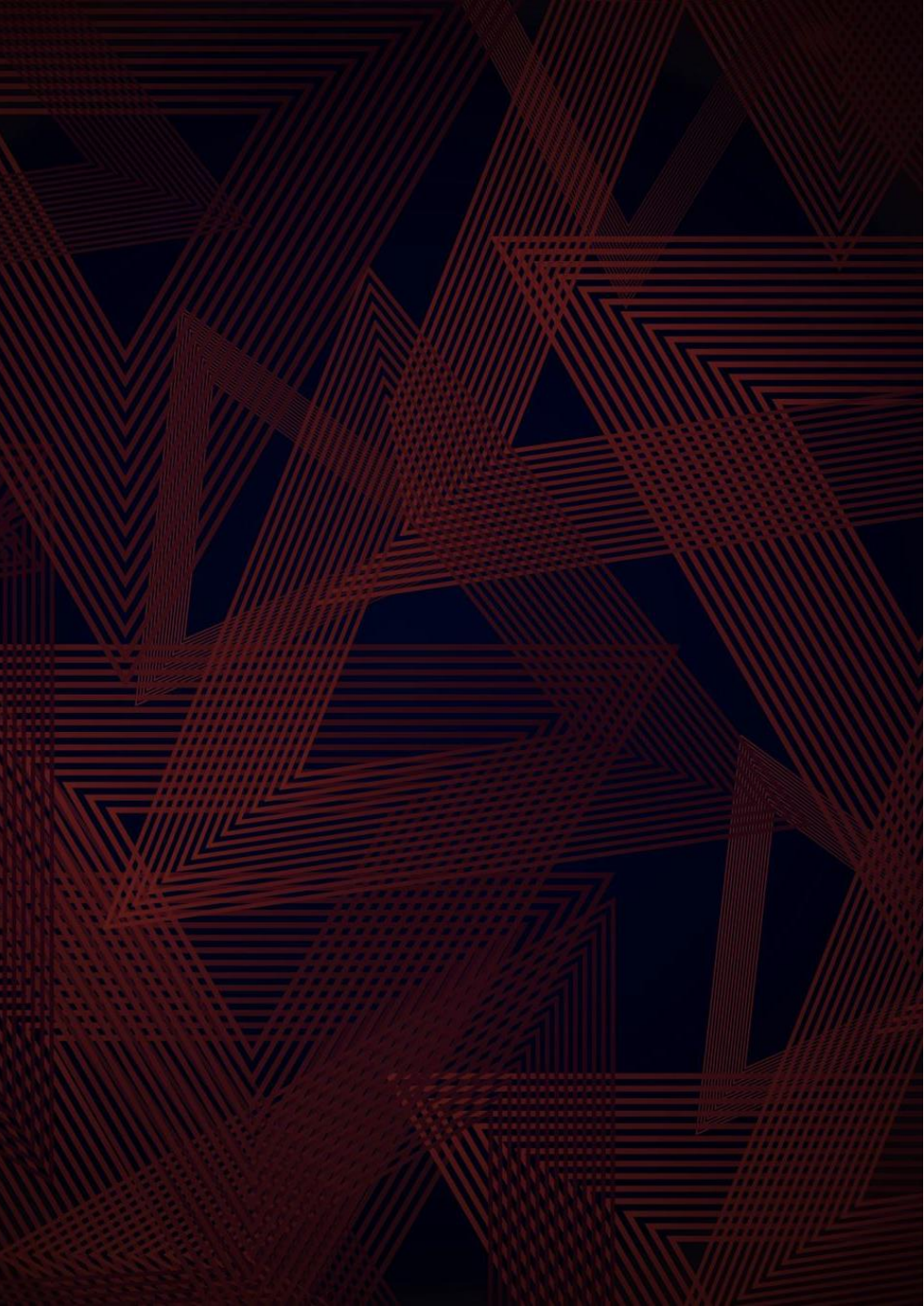
## 8. Calcular la Matriz de Confusión

```
1 pred_labels = np.argmax(predictions, axis=1)
2 true_labels = test_labels.flatten()
3
4
5 conf_matrix = confusion_matrix(true_labels, pred_labels)
6
7 plt.figure(figsize=(10,8))
8 sns.heatmap(conf_matrix, annot=True, fmt="d",
9             xticklabels=class_names,
10            yticklabels=class_names,
11            cmap="Blues") # Uso de la paleta de colores azules
12 plt.title('Matriz de Confusión \n')
13 plt.ylabel('Etiqueta Verdadera')
14 plt.xlabel('Etiqueta Predicha')
15 plt.show()
16
```



CONFUSION



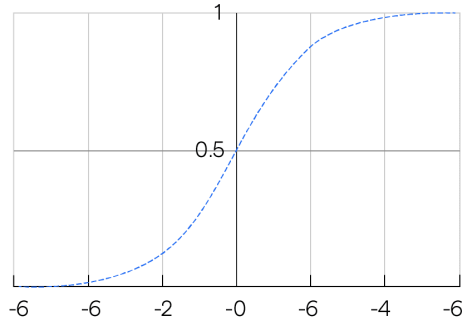


# Ejercicio 1: Modificar la Arquitectura de la Red

- **Objetivo:** Familiarizarse con el impacto de la arquitectura de la red en el rendimiento del modelo.
- **Tarea:** Añada una capa convolucional adicional con 128 filtros, un kernel de tamaño (3,3) y activación ReLU.
- ¿Cómo afecta esta adición al número de parámetros entrenables y al rendimiento del modelo?

### Sigmoid Function

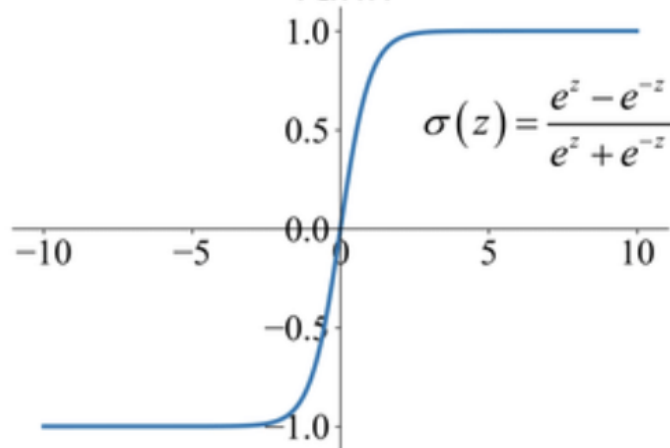
$$f(x) = \frac{1}{1 + e^{-fx}}$$



## Ejercicio 2: Experimentar con Diferentes Funciones de Activación

- **Objetivo:** Entender cómo las funciones de activación afectan el aprendizaje del modelo.
- **Tarea:** Cambie la función de activación de las capas densas de ReLU a **tanh** o **sigmoid**.
- **Pregunta de reflexión:** ¿Qué efecto tienen estas funciones de activación en la convergencia del modelo durante el entrenamiento?

### Tanh



## Ejercicio 3: Ajustar los Parámetros del Optimizador

- **Objetivo:** Comprender la influencia de los parámetros del optimizador en el entrenamiento del modelo.
- **Tarea:** Modifique los parámetros del optimizador Adam, como el learning rate (tasa de aprendizaje), para observar cómo afectan al proceso de entrenamiento. Pruebe con learning rates de 0.01 y 0.0001.
- ¿Cómo varía la velocidad de convergencia del modelo con diferentes tasas de aprendizaje?